

Introduction

Technologies for Advanced Programming (TAP) 2026/2027

Salvo Nicotra

Technologies for Advanced Programming (TAP)

Salvo Nicotra

Course details

Lessons

- 24 lessons: 3 hours each, Thursday and Friday, 15:00–18:00
- Hands-on approach: bring your laptop and be proactive in class
- 2–3 invited seminars from industry and research
- Bring questions and project ideas to class

Office hours

- By appointment, in person or on Teams
- salvatore.nicotra1@unict.it

Course community

Telegram course group

The telegram group is used for announcements, questions, and discussions. Students are encouraged to join the group and participate actively.

Note

New 2027: as part of the course we will develop a TAP agent to help students with questions and exercises. The agent will be available in the Telegram group.

[Join the TAP group](#)

GitHub organization

The GitHub organization is used to host the course repositories, including code, notebooks, and exercises. The projects of previous years are available are also forked in the organization.

[Tap Unict](#)

Introductions

Before introduce the course details, let's do a quick introduction round.

- I will launch a ball in the class, and each of you will have a chance to introduce yourself.
- Share your name, background, why you are taking this course, and what you expect to learn from this course.
- Then, please throw the ball to another student and repeat the process until everyone has introduced themselves.

Course goals

Official 2026/2027 syllabus TO BE UPDATED

By the end of the course, students should be able to design and implement data-intensive applications using modern tools and architectures. The course combines theory, demonstrations, and practical work across:

- Real-time data ingestion and stream processing, including Kafka and Spark.
- Containerized applications and scalable data platforms, from modern data stacks to Lakehouse architectures.
- Data storage, quality, governance, and observability.
- Search, vector databases, and AI applications such as Retrieval-Augmented Generation (RAG) and agents.

TAP pillars

Memes



imgflip.com

Caution

Gen AI is great, but is not you, avoid ugly standard ai-generated memes

Solve problems



<https://bit.ly/theonetap>



Tip

Open with mobile phones

Love for technology

- This is a course about technology, how to choose, adopt and evaluate it.
- There is no a single better technology, there are only better technologies for specific use cases.
- For the purposes of this course, we will focus on a “standard” modern data stack to build streaming data pipelines, but the spirit of the course is to learn how to decide which technology to use for a specific problem, taking into account all relevant aspects, including scalability, sustainability, security, and ease of adoption.



NicsMeme

Topics: problems to solve

A data-driven application needs to handle diverse, fast-changing **data** into **information** people can use safely and reliably.

1. **Acquire and integrate:** Bring together data from different sources, formats, and APIs—by batch or as events arrive.
2. **Move and process:** Validate, transform, combine, and analyze data at the scale and latency the use case needs.
3. **Store and manage:** Keep data durable and discoverable as schemas and requirements evolve; consider quality, access, and lineage.
4. **Find and use:** Support analytics, visualization, search, machine learning, and AI applications with relevant, trustworthy data.

Tip

What data is needed, how quickly, at what scale, and with which quality, privacy, reliability, and cost constraints?

From problems to an evolving ecosystem

The four problems on the previous slide (acquire, move, store, find) haven't changed, but *how* systems solve them has evolved as data volume, velocity, and variety grew.

Two levels

- **Architecture patterns:** how acquisition, processing, and storage fit together, and how that fit has changed over time.
- **Per-topic technologies:** real-world tools for each stage of the pipeline.

The story we'll trace

1. Batch vs. streaming: a trade-off between freshness and simplicity.
2. Lambda architecture: an early fix that runs both, at a cost.
3. Lakehouse: a more recent fix that removes that cost.

Batch vs. streaming processing

Two complementary ways to move and process data, differing in when work happens and how fresh the results are.

Batch processing

- Process bounded, already-collected data on a schedule (hourly, nightly, ...).
- *You gain:* high throughput, simple to reason about, easy to retry/reprocess from scratch.
- *You pay:* latency: results reflect data as of the last run, not right now. (and also costs)
- Example: nightly re-index of course documents, data warehouse ETL, or a nightly report of the day's sales.

Stream processing

- Process unbounded events as they arrive, one at a time or in small windows.
- *You gain:* low latency: results available continuously, close to real time.
- *You pay:* complexity: ordering, late data, state, and exactly-once semantics.
- Example: an incoming Telegram message triggering the assistant.

Tip

Does the use case need an answer now, or is a periodic refresh good enough? Many real systems need *both* ?

The Lambda architecture (2011)

The problem: early big-data systems needed both a correct, complete historical view *and* a low-latency view of recent events—batch alone was too slow, streaming alone was hard to make fully correct.

Three layers

- **Batch layer:** recomputes precise, complete views over all historical data (e.g., with Spark/Hadoop).
- **Speed layer:** processes recent events immediately (e.g., with Kafka Streams), approximating the batch view until it catches up.
- **Serving layer:** merges batch and speed views to answer queries.

The fix (proposed by Nathan Marz creator of Apache Storm): run batch and streaming *side by side*, over the same raw data, and merge their results.

Trade-off

- *You gain:* correctness (batch) plus freshness (speed) for the same data.
- *You pay:* two codebases/pipelines implementing similar logic twice, which must be built, tested, and kept consistent.
- This cost is the reason it was later challenged and alternatives emerged.

Note

Jay Kreps (creator of Apache Kafka and co-founder of Confluent) in [“Questioning the Lambda Architecture”](#) (O’Reilly Radar, 2014) proposes the single-pipeline **Kappa architecture** as a response

Evolution toward Lakehouse architectures

Jay Kreps' critique motivated simplifying Lambda's two pipelines into one. Storage evolved the same way, to remove duplication at the data layer too.

Data warehouse:

Structured, curated data; strong schema and query performance.

Costly to scale, and not suited to raw/unstructured or ML data.

Data lake:

Raw files at scale, any format, low storage cost (e.g., HDFS, MinIO, RustFS).

Weak consistency and governance: easy to turn into a “data swamp”.

Lakehouse:

Table formats (Apache Iceberg, Delta Lake) add ACID transactions, schema evolution, and time travel on top of lake storage.

The same table can be read by a batch job and written by a streaming job—one copy of the data, not two pipelines.

Solves the *storage* side of the Lambda trade-off—but storage is only one stage of the pipeline.

Tip

The bigger picture:** acquiring, moving, searching, and using data each still raise their own design questions

**Technologies: tools to fix the
problem**

Data Ingestion

Bring data from applications, logs, and APIs into the pipeline, by batch or as events arrive.

Technologies

- Apache Flume
- Apache NiFi
- Logstash
- Fluentd
- n8n

Applications

- Collect and reliably move events from application logs into storage.
- Route, transform, and monitor data flows across heterogeneous systems.
- Parse, enrich, and route application and infrastructure logs.
- Aggregate logs from distributed services and forward them to a searchable store.
- Connect APIs and automate operational workflows with low-code orchestration.

Data Streaming

Buffer, distribute, and process events in real time, connecting producers and consumers asynchronously.

Technologies

- Apache Kafka (Connect, Streams, Schema Registry)
- Confluent
- Redpanda
- Apache Pulsar
- NATS

Applications

- Buffer and distribute events for decoupled services and real-time pipelines.
- Move records between Kafka and external systems without custom producer/consumer code.
- Implement lightweight, stateful stream transformations directly on Kafka topics.
- Enforce and evolve message schemas; a data dictionary for event streams.
- Operate Kafka-based pipelines with managed connectors and platform tooling.

Data Processing

Transform, validate, and analyze data at the scale and latency the use case needs, in batch or streaming.

Technologies

- Apache Spark (with Spark NLP)
- Apache Flink
- Ray
- DuckDB

Applications

- Transform large datasets in batch and streaming workloads; process Lakehouse tables with Iceberg.
- Process and analyze text at scale within Spark pipelines, including LLM-assisted tasks.
- Alternative to Spark; process unbounded streams with low latency and exactly-once semantics.
- Alternative to Spark/Flink; scale Python workloads, including ML model training and inference.
- Alternative to Spark; run SQL queries on local or in-memory datasets, including Parquet and Arrow files.

Real Time Analytics

Run low-latency aggregate queries directly over streamed or ingested events, without a separate batch step.

Technologies

- ClickHouse

Applications

- Run low-latency aggregate queries directly over streamed or ingested events for dashboards and monitoring.

Data Storage

Keep data durable and discoverable as schemas and requirements evolve, from distributed file systems to object storage.

Technologies

- HDFS
- MinIO
- RustFS

Applications

- Store large datasets across a cluster; understand the storage foundations underlying modern Lakehouse designs.
- Provide S3-compatible storage for Lakehouse data files and course documents.
- Alternative to MinIO; S3-compatible object storage implemented in Rust.

Data Lake

Add table-format guarantees-schema evolution, ACID transactions, and time travel-on top of files in object storage.

Technologies

- Apache Iceberg
- Delta Lake

Applications

- Add schema evolution, snapshots, and time travel to analytical tables stored in object storage and processed with Spark.
- Alternative to Iceberg; ACID transactions, schema evolution, and time travel on tables stored in object storage.

Search & Retrieval

Index and retrieve data by keyword or meaning, powering operational search and RAG-based question answering.

Technologies

- Elasticsearch
- OpenSearch

Applications

- Index logs and documents; also used as a vector database to retrieve relevant sources for RAG-based question answering.
- Search application data; also used as a vector database for semantic search and RAG.

Visualization & Monitoring

Explore indexed data and monitor service and pipeline health through dashboards.

Technologies

- Kibana
- Grafana

Applications

- Explore indexed data and build dashboards for logs, events, and metrics.
- Monitor service and pipeline metrics and investigate operational issues.

Data Governance

Catalog datasets, track lineage, and surface data ownership and quality signals across the pipeline.

Technologies

- OpenMetadata

Applications

- Catalog datasets, track lineage, and surface data ownership and quality signals.

AI / Generative

Serve and orchestrate language models to generate grounded answers, with tools and citations, for the course assistant.

Technologies

- Hugging Face
- vLLM
- Ollama
- LangChain

Applications

- Discover and reuse language models for grounded answers, including models with tool-calling support where available.
- Serve open-weight LLMs efficiently at scale for the course assistant's answer generation.
- Run and prototype with LLMs locally for development, testing, and small-scale demos.
- Wire retrieval, tools, and an LLM into the assistant's answer/no-answer, citation, and escalation workflow.

Platform & Infrastructure

Package, deploy, and scale the services that make up the pipeline and the assistant.

Technologies

- Docker
- Kubernetes

Applications

- Package and run reproducible data services and processing jobs.
- Deploy, scale, and recover containerized data services across a cluster.

Technology map: putting it all together

One plausible way these technologies connect in a data pipeline, from ingestion through processing, storage, search, and the AI assistant.

Ingest: NiFi, Flume, Logstash, Fluentd, n8n

Stream: Kafka, Kafka Connect, Kafka Streams, Schema Registry, Confluent (alternatives: Redpanda, Apache Pulsar, NATS)

Process: Spark, Spark NLP (alternatives: Flink, Ray, DuckDB)

Analyze: ClickHouse

Store: HDFS, MinIO, RustFS

Data Lake: Apache Iceberg, Delta Lake

Search: Elasticsearch, OpenSearch (also as vector databases)

Operate & govern: Kibana, Grafana, OpenMetadata

AI / Generative: Hugging Face, vLLM, Ollama, LangChain

Deployed and orchestrated with **Docker** and **Kubernetes**.

Applications: what we build

The same building blocks can support different applications. Requirements determine which technologies we combine.

Real-time data products

Event-driven analytics and dashboards turn streams into timely, searchable insights.

Analytics and machine learning

Explore data, classify or cluster it, analyze text, and evaluate whether model results are useful.

AI-assisted information access

Search approved course documents and retrieve relevant passages. Answer an addressed Telegram question with source citations; decline unsupported questions and hand off when uncertain.

Tip

The course project demonstrates the integration of ingestion, processing, search, and AI together while considering privacy and data quality.

